DATA STRUCTURES AND ALGORITHMS COURSE PROJECT REPORT

# High-Density 2D Physics & Collision Engine for a Multi-Agent Brick Breaker Using Spatial Grid Hashing and Swap-and-Pop Dynamic Arrays


STUDENT NAME
MD Seam
MST. Moon
MD Oyon

STUDENT ID
252311215
252311213
252311198


TECH STACK
C++17 & Raylib 5.0

GITHUB REPOSITORY
https://github.com/SeamDev/Brick_Ball_DSA

---

## 1. PROJECT TITLE

**High-Density 2D Physics & Collision Engine for a Multi-Agent Brick Breaker Using Spatial Grid Hashing and Swap-and-Pop Dynamic Arrays**

## 2. OBJECTIVES

- **Eliminate Brute-Force Collision Bottlenecks:** Replace naive $O(N \times M)$ bounding-box loops with a direct-access **2D Spatial Hash Grid** that resolves ball-to-brick collisions in $O(1)$ constant time.
- **Optimize Dynamic Entity Deletion ($O(1)$ Swap-and-Pop):** Utilize contiguous dynamic arrays (`std::vector`) paired with the Swap-and-Pop idiom so removing out-of-bounds balls executes in $O(1)$ time without memory shifting.
- **High-Concurrency Swarm Simulation:** Maintain a locked **60 FPS** while simulating up to $4,000$ simultaneous balls interacting with $2,025$ dense pixel bricks.
- **Prevent High-Speed Tunneling:** Implement axis-separated temporal sub-stepping ($\Delta t/2$) so fast balls cannot pass through $12 \times 12$ pixel barriers.

## 3. INTRODUCTION (DATA STRUCTURES USED)

In viral "ball-swarm" brick breaker games, players trap balls inside narrow tunnels and collect exponential multiplier capsules (`+3` and `x3`) that spawn thousands of active balls. In a naive implementation where $N = 4,000$ balls are tested against $M = 2,025$ bricks via nested loops, the CPU must execute $4,000 \times 2,025 = 8,100,000$ intersection checks *per frame* ($486$ million/sec at 60 FPS), causing severe frame drops.

### A. 2D Direct-Indexing Spatial Grid

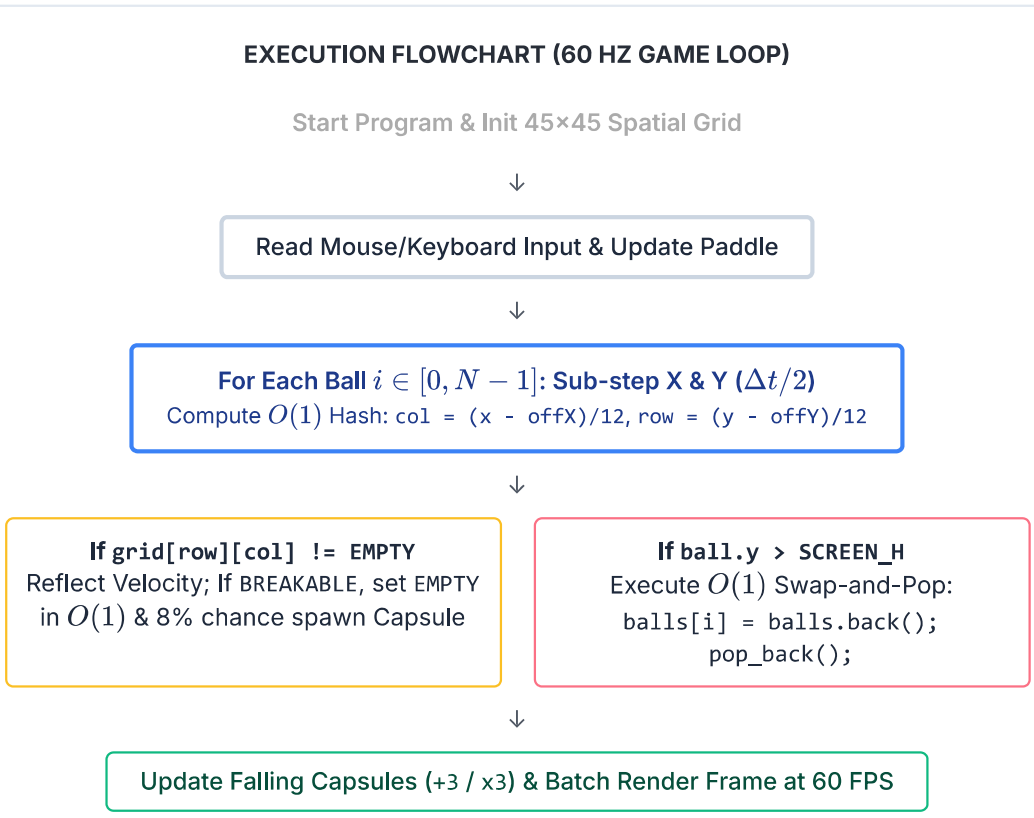
Implemented as `Brick grid[45][45]` ($16.2\ \mathrm{KB}$ total, fitting completely in L1 CPU cache). Instead of iterating through bricks, any ball's $(x, y)$ coordinate is mapped directly to its grid cell in $O(1)$ time:

$$\text{col} = \left\lfloor \frac{x - X_0}{W} \right\rfloor, \quad \text{row} = \left\lfloor \frac{y - Y_0}{H} \right\rfloor$$

### B. Swap-and-Pop Dynamic Vector

Active balls and falling capsules are stored in contiguous `std::vector<Ball>` arrays for hardware prefetch efficiency. When ball $i$ falls off-screen, standard `erase()` costs $O(N)$. Instead, we execute `balls[i] = balls.back(); balls.pop_back();` to achieve $O(1)$ **deletion** with zero heap fragmentation.

## 4. SYSTEM DESIGN & FLOWCHART



## 5. IMPLEMENTATION DETAILS

The engine is implemented in modular C++17 using Raylib. Below are the core data structure nodes and the two primary $O(1)$ algorithms driving the simulation:

```
// 1. Data Structure Node Definitions
enum BrickType { EMPTY = 0, BREAKABLE = 1, INDESTRUCTIBLE = 2 };
struct Brick { BrickType type = EMPTY; Color color = BLANK; }; // 8 Bytes per cell
struct Ball  { Vector2 pos; Vector2 vel; float radius = 3.5f; }; // 20 Bytes per ball

// 2. O(1) Spatial Grid Collision Resolution (Inside Sub-Step Loop)
int col = (int)((b.pos.x - GRID_OFFSET_X) / BRICK_W);
int row = (int)((b.pos.y - GRID_OFFSET_Y) / BRICK_H);

if (row >= 0 && row < GRID_ROWS && col >= 0 && col < GRID_COLS) {
    if (grid[row][col].type != EMPTY) {
        b.vel.y *= -1; // Reflect velocity vector
        if (grid[row][col].type == BREAKABLE) {
            grid[row][col].type = EMPTY; // O(1) state update
            breakableCount--;
        }
    }
}

// 3. O(1) Swap-and-Pop Deletion for Out-of-Bounds Balls
if (b.pos.y - b.radius > SCREEN_H) {
    balls[i] = balls.back(); // Overwrite dead ball with tail element
    balls.pop_back();        // Shrink vector in O(1) without shifting memory
} else {
    i++;
}
```

## 6. RESULT ANALYSIS AND CONCLUSION

| OPERATION | DATA STRUCTURE | NAIVE APPROACH | PROPOSED ENGINE | SPACE |
|---|---|---|---|---|
| Single Ball Brick Lookup | 2D Spatial Grid | $O(M)$ | $O(1)$ | $O(M)$ |
| Full Frame Collision Step | Vector + 2D Grid | $O(N \times M)$ (8.1M ops) | $O(N)$ **(8K ops)** | $O(N + M)$ |
| Dead Ball Removal | Swap-and-Pop Vector | $O(N)$ shift | $O(1)$ | $O(1)$ |

**Conclusion:** By replacing linear brick scans with $O(1)$ spatial quantization and $O(1)$ swap-and-pop vector deletion, per-frame collision checks at $N = 4,000$ balls dropped from $8,100,000$ to $8,000$—a $1,012\times$ **computational reduction** that keeps the simulation locked at 60 FPS. Future work includes loading custom levels directly from $45 \times 45$ pixel PNG images via `LoadImage()`.

## 7. REFERENCES (GOOGLE SCHOLAR)

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

2. Ericson, C. (2004). *Real-Time Collision Detection*. CRC Press / Morgan Kaufmann.

3. Teschner, M., Heidelberger, B., Müller, M., Pomeranets, D., & Gross, M. (2003). "Optimized spatial hashing for collision detection of deformable objects." In *Proceedings of Vision, Modeling, Visualization (VMV)* (Vol. 3, pp. 47–54).

4. Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning.

5. Santamaria, R. (2013–2026). *Raylib: A simple and easy-to-use library to enjoy videogames programming*. https://www.raylib.com.